Report

on

Virtual Machine Setup, Auto-Scaling, and Security Configuration on Google Cloud Platform (GCP)

Course Code

CSL7510

submitted by

Ankit Kumar Chauhan (M23CSA509)

M.Tech- Artificial Intelligence (Executive)

Course instructor

Dr. SUMIT KALRA

Department of Computer Science and Engineering

Indian Institute of Technology Jodhpur
NH 65, Nagaur Road, Karwar,
Jodhpur 342030, INDIA

1. Introduction

The purpose of this document is to provide a detailed guide for deploying a scalable and secure infrastructure on Google Cloud Platform (GCP). The focus is on automating the creation of virtual machines (VMs), configuring dynamic auto-scaling policies to efficiently handle workload variations, and implementing robust security measures such as custom firewall rules, and Identity and Access Management (IAM) roles. The setup is ensured to be consistent, repeatable, and efficient by using an automated deployment script. Moreover, the guide comprises architecture visualization, testing procedures for auto-scaling and security configurations, and steps for resource cleanup to optimize cost management. The objective of this document is to empower users with a clear and practical approach to setting up and managing the GCP infrastructure.

2. Step-by-Step Instructions for Implementation

- ***Step 1: Script-Based Deployment of GCP Infrastructure***

❖ Prerequisites

- Ensure that the Google Cloud SDK is installed. If not, the script will take care of the installation.
- Authenticate the GCP CLI using *gcloud auth login*. If not, the script will ask for the same.

❖ Execution of the Script

1. Download the provided script file i.e. assignment2_gcp-auto-scaling.sh.
2. Make the script executable by running:

   chmod +x assignment2_gcp-auto-scaling.sh

3. Execute the script:

   ./assignment2_gcp-auto-scaling.sh

❖ What the Script Does?

1. **Create an Instance Template**: Configures a template with the required settings (Fig. 1), which includes a startup script to install Apache and serves a test webpage.

```
# ---- DEPLOY INFRASTRUCTURE ----
# Create instance template
echo "Creating instance template..."
gcloud compute instance-templates create "$INSTANCE_TEMPLATE" \
  --machine-type=e2-micro \
  --image-family=ubuntu-2004-lts \
  --image-project=ubuntu-os-cloud \
  --tags=http-server,ssh-my-ip \
  --service-account="$SERVICE_ACCOUNT_EMAIL" \
  --scopes=cloud-platform \
  --metadata=startup-script='#!/bin/bash
    sudo apt update -y
    sudo apt install -y apache2
    sudo systemctl start apache2
    sudo systemctl enable apache2
    echo "Hello from $(hostname)" > /var/www/html/index.html'
```

Fig. 1. Configuration settings for instance template

2. **Create a Managed Instance Group**: The template is used to deploy instances, which enable dynamic scaling. The settings for the managed instance group are shown in Fig. 2.

```
# Create managed instance group with auto-scaling
echo "Creating managed instance group..."
gcloud compute instance-groups managed create "$INSTANCE_GROUP" \
  --base-instance-name=web-instance \
  --template="$INSTANCE_TEMPLATE" \
  --size="$MIN_REPLICAS" \
  --zone="$ZONE"
```

Fig. 2. Configuration settings for managed instance group

3. **Create Auto-Scaling Policies**: Sets a target CPU utilization of 60%, with a minimum of one instance and a maximum of five instances. In Fig. 3, the policies for auto-scaling are shown.

```
# Configure auto-scaling policy
echo "Configuring auto-scaling..."
gcloud compute instance-groups managed set-autoscaling "$INSTANCE_GROUP" \
  --zone="$ZONE" \
  --min-num-replicas="$MIN_REPLICAS" \
  --max-num-replicas="$MAX_REPLICAS" \
  --target-cpu-utilization="$TARGET_CPU_UTILIZATION" \
  --cool-down-period=120

gcloud compute instance-groups managed set-named-ports "$INSTANCE_GROUP" \
  --zone="$ZONE" \
  --named-ports=http:80
```

Fig. 3. Configuration of auto-scaling policies

4. **Create Firewall Rules**: Configures rules to allow HTTP (tcp:80), SSH (tcp:22) from specified IP, and health check traffic from Google IP ranges. The firewall rules created using the code below are shown in Fig. 4.

```
# Configure firewall rules
echo "Setting up firewall rules..."
# Allow HTTP traffic
gcloud compute firewall-rules create allow-http \
  --allow=tcp:80 \
  --source-ranges=0.0.0.0/0 \
  --target-tags=http-server

# Allow SSH only from your public IP
gcloud compute firewall-rules create allow-ssh-my-ip \
  --allow=tcp:22 \
  --source-ranges="$MY_IP" \
  --target-tags=ssh-my-ip

# Allow health checks from Google IP ranges
gcloud compute firewall-rules create allow-health-check \
  --allow=tcp:80 \
  --source-ranges="130.211.0.0/22,35.191.0.0/16" \
  --target-tags=http-server
```

Fig. 4. Creation of firewall rules

5. **Creation of a Load Balancer**: Creates a load balancer to constantly monitor the health of all the virtual machine and distributes the load among them. The load balancer setup

```
# ---- LOAD BALANCER SETUP ----
echo "Setting up load balancer..."
# Create a health check
gcloud compute health-checks create http web-health-check \
  --port=80 \
  --check-interval=10s \
  --timeout=5s \
  --healthy-threshold=2 \
  --unhealthy-threshold=3

# Create a backend service
gcloud compute backend-services create web-backend-service \
  --protocol=HTTP \
  --port-name=http \
  --health-checks=web-health-check \
  --global

# Add the instance group to the backend service
gcloud compute backend-services add-backend web-backend-service \
  --instance-group="$INSTANCE_GROUP" \
  --instance-group-zone="$ZONE" \
  --global

# Create a URL map
gcloud compute url-maps create web-url-map \
  --default-service=web-backend-service

# Create a target HTTP proxy
gcloud compute target-http-proxies create web-http-proxy \
  --url-map=web-url-map

# Create a global forwarding rule
gcloud compute forwarding-rules create web-forwarding-rule \
  --global \
  --target-http-proxy=web-http-proxy \
  --ports=80
```

Fig. 5. Configuration for Load Balancer

configuration is shown in above Fig. 5.

6. **Create Identity and Access Management (IAM) Roles**: Defines roles such as Compute Viewer and Compute Instance Admin to restrict and manage access. I have created a user named *ankichauhan@gmail.com* with viewing permission only, whereas created user named m23csa509@iitj.ac.in with instance admin role.

```
# ---- IAM ROLES ----
echo "Assigning IAM roles..."
# Assign Compute Viewer role to a user
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="user:ankichauhan@gmail.com" \
  --role=roles/compute.viewer

# Assign Compute Instance Admin role to a user (optional)
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="user:$USER_EMAIL" \
  --role=roles/compute.instanceAdmin.v1
```

Fig. 6. Configuration for setting up of different roles

➢ *Step 2: Key Configuration Details*

❖ Auto-Scaling Policies

- **Minimum Instances**: 1
- **Maximum Instances**: 5
- **Target CPU Utilization**: 60%
- **Cooldown Period**: 120 seconds

❖ Firewall Rules

- Allow HTTP traffic (tcp:80) from all sources.
- Restrict SSH access (tcp:22) to a specific IP range (configured via MY_IP in the script).
- Allow health check traffic from Google IP ranges.

❖ IAM Role Assignments

- Assigns Compute Viewer and Compute Instance Admin roles to specified users to regulate access and permissions.

❖ Load Balancer Configuration

- Sets up a health check to monitor the backend instance health.
- Configures a backend service that is linked to the managed instance group.
- Establishes a URL map, HTTP proxy, and global forwarding rule for traffic routing.

➢ *Step 3: Testing the Setup*

❖ Testing Auto-Scaling

1. **SSH into an Instance**: The command to ssh into creating a VM instance is given below. Fig. 7 shows ssh into the created VM.

gcloud compute ssh my-instance --zone=us-central1-a

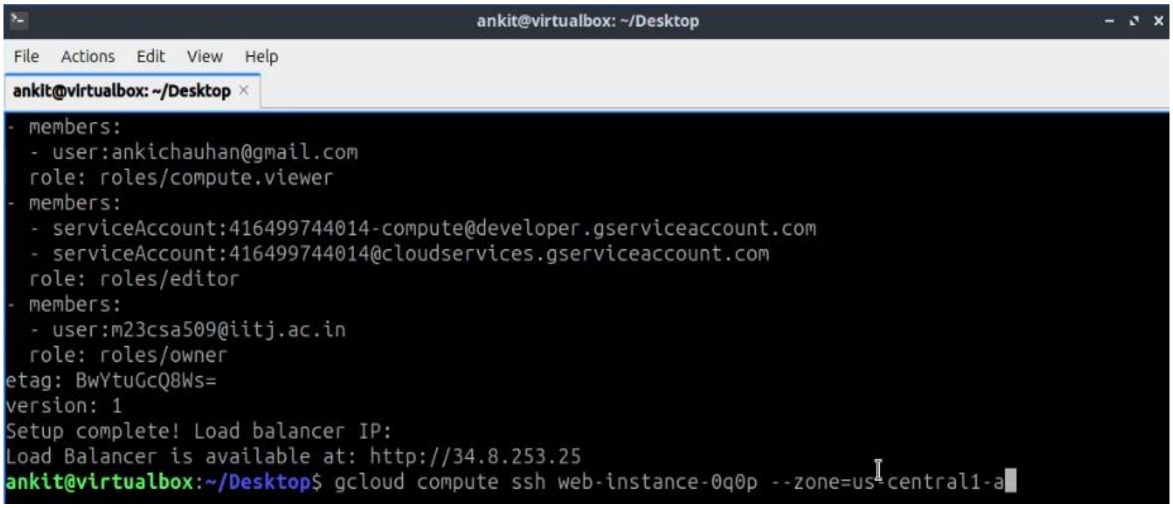

Fig. 7. Logging into the created Virtual Machine in GCP

The successful login into the VM is shown in Fig. 8 below.

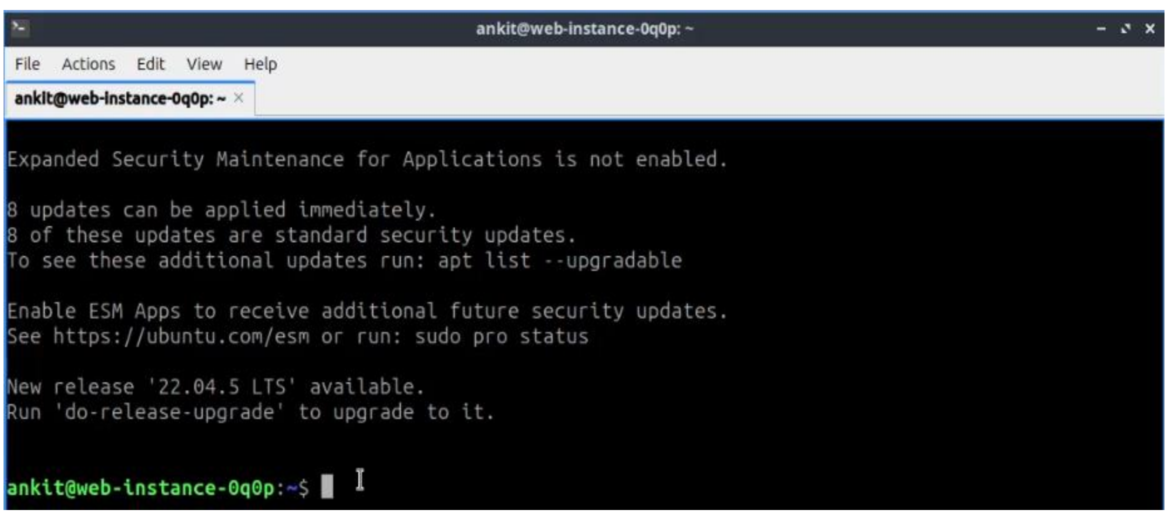

Fig. 8. Logged into the Virtual Machine

2. **Install Stress Utility:**

sudo apt update && sudo apt install stress

3. **Trigger Auto-Scaling**: Run the following command to stress the CPU (Fig. 9):

stress --cpu 2 --timeout 120

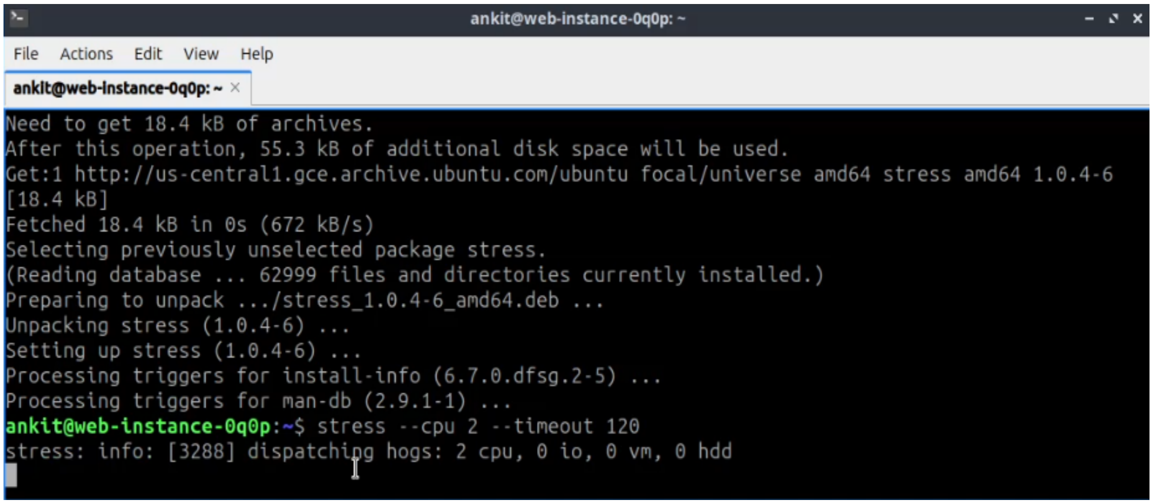

Fig. 9. Putting load on CPU of VM

4. **Check Auto-Scaling Behavior**: The behavior of auto scaling is shown in below Fig. 10. As we can see that five virtual machines were created using the auto scaling policy.

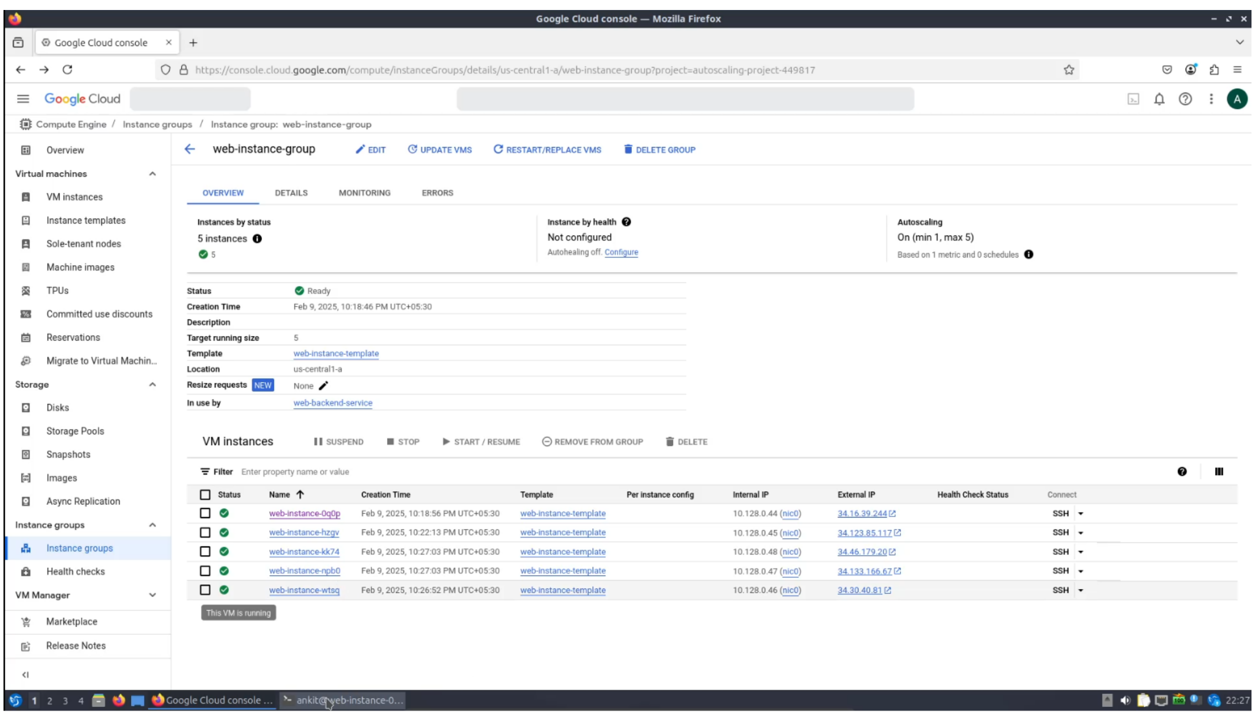

Fig. 10. Multiple VMs created using auto-scaling

❖ Check Load Balancer Functioning

- The load balancer is running flawlessly as shown in below Fig. 11. Here, the load balancer distributes the load among active VMs, and we can clearly see that request is being served by different VMs.

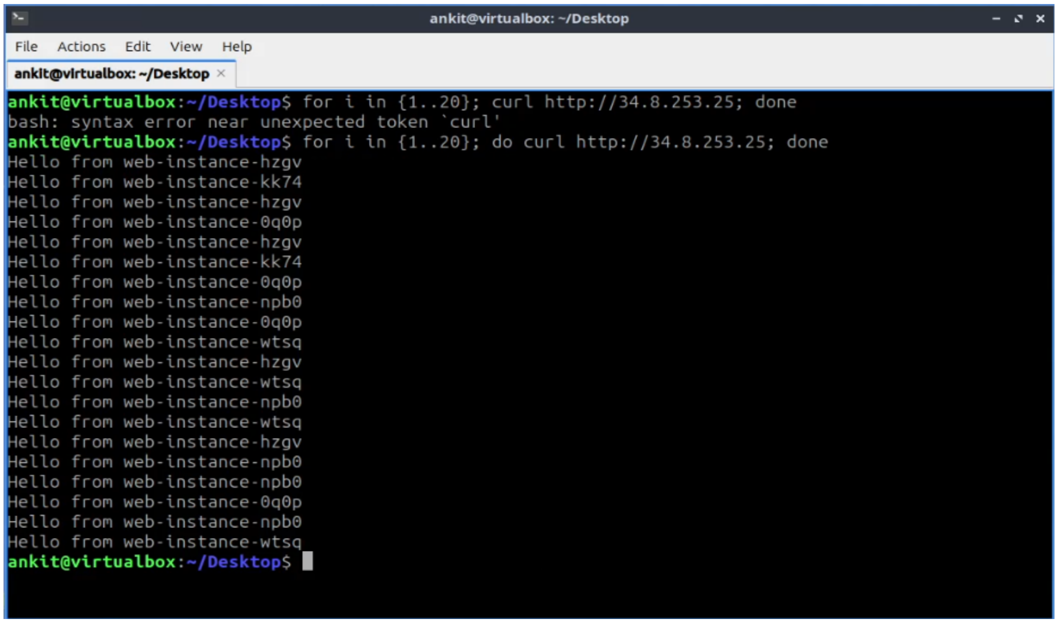

Fig. 11. Service requests served by different VMs using load balancer

❖ Checking Firewall Rules

1. **List Firewall Rules**: The bash script created firewall rules as shown in Fig. 12 below.

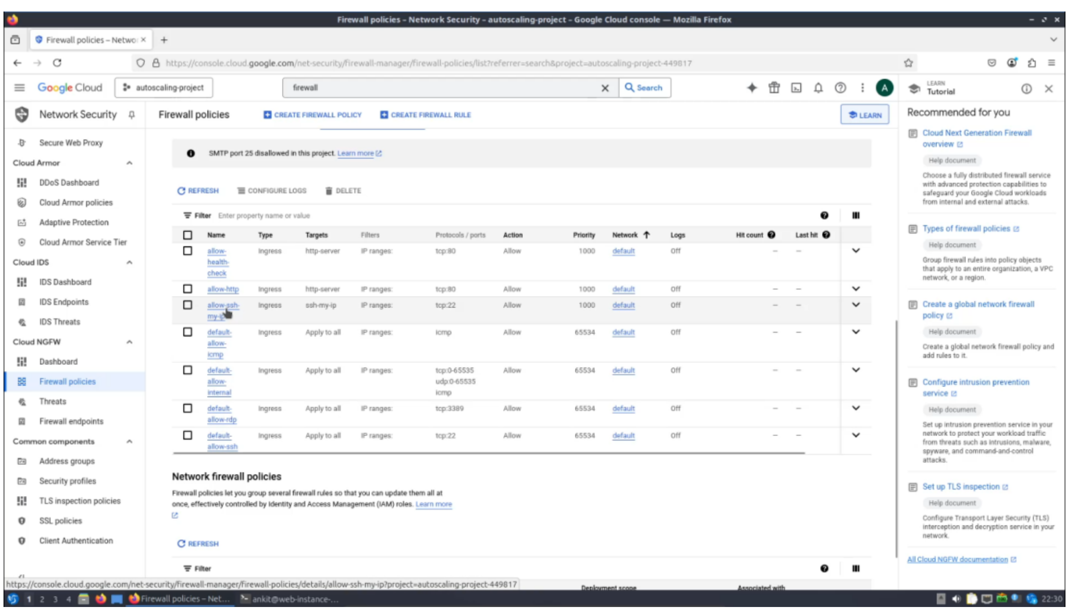

Fig. 12. Firewall rules

2. **Verify the rules:** As we have already logged in to the VM using ssh, we can say that the firewall is allowing traffic at port 22. Now let's see if the firewall allows traffic at port 22 through the load balancer. Port 22 is being hit by the load balancer, as shown in Fig. 13.

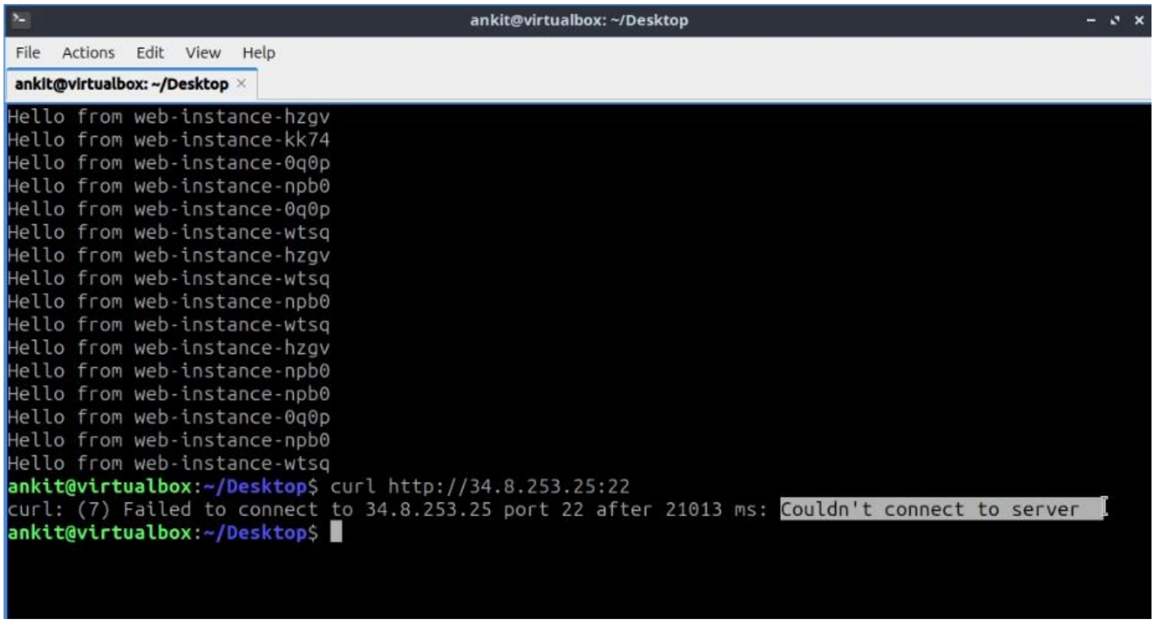

Fig. 13. Firewall didn't allow traffic at port 22 for the load balancer

This is in accordance to the firewall rules we created, as traffic to the load balancer only allows for port 80.

❖ Checking IAM Roles

1. **View IAM Roles: In** Fig. 14, the IAM roles have been created. We can see that user ankichauhan@gmail.com have 'viewer' role and the user m23csa509@iitj.ac.in have owner and 'Instance Admin' role.

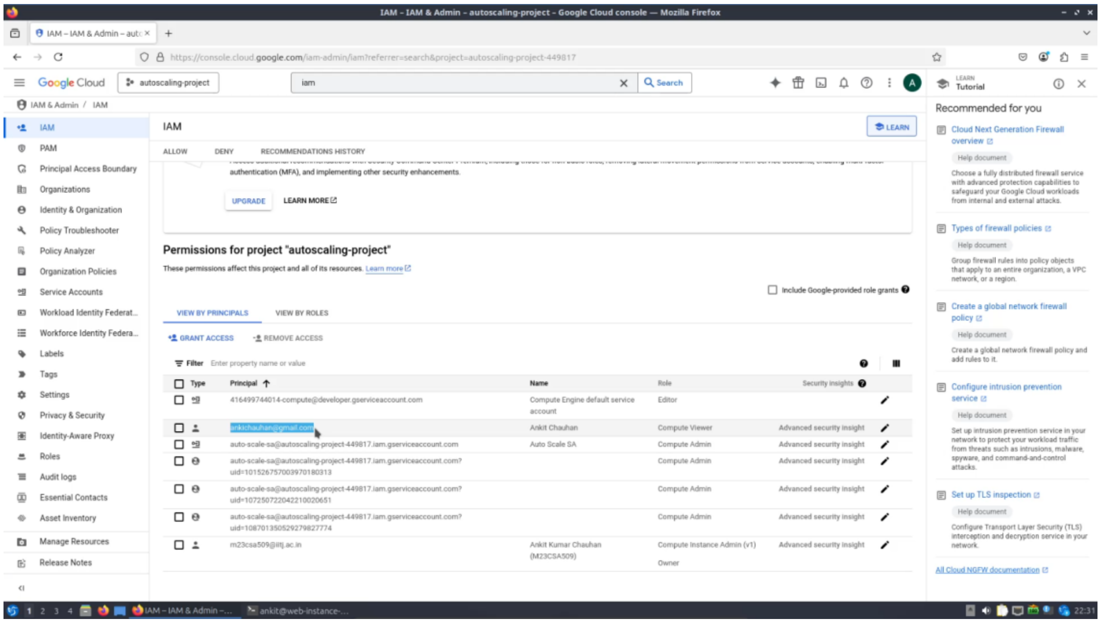

Fig. 14. IAM roles

2. **Test User Permissions**:

- Log in with the user credentials: Use below command. Log in using the user ankichauhan@gmail.com .

gcloud auth login

- Perform actions with permitted permissions (e.g., list instances): Use below command. The current instances after scaling down are shown in Fig. 15.

gcloud compute instances list --project=autoscaling-project-449817

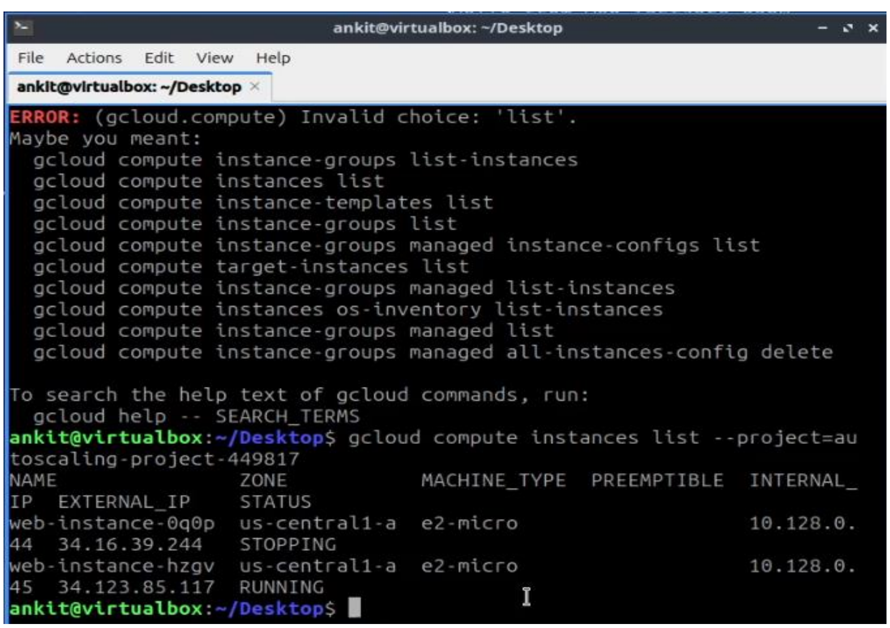

Fig. 15. Current active VMs

- Attempt actions outside of permissions (e.g., stop an instance): The command is given below. We can replace the placeholder for instance name and zone. Below Fig. 16, it shows that stopping of the VM failed because the user does not have admin privileges. The user who logged in has only viewing permission. Thus, IAM roles are working correctly.

  gcloud compute instances stop <INSTANCE_NAME> --zone="$ZONE"

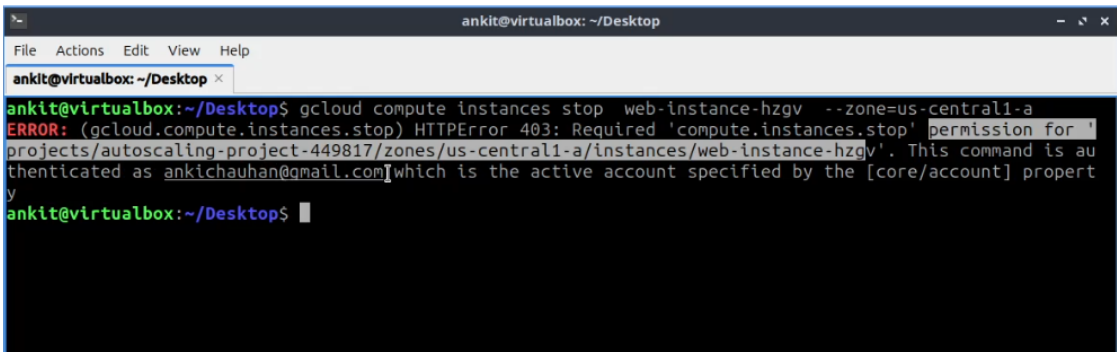

Fig. 16. Stopping of VM failed (no Instance Admin role)

- ***Step 4: Cleanup***

  To save costs, delete the created resources:

1. **Make the script executable by running:**

   chmod +x assignment2_gcp_delete.sh

2. **Run the Delete Script**: Execute the provided script to remove all resources:

   . / assignment2_gcp_delete.sh

   In Fig. 17, all resources created in GCP are deleted successfully.

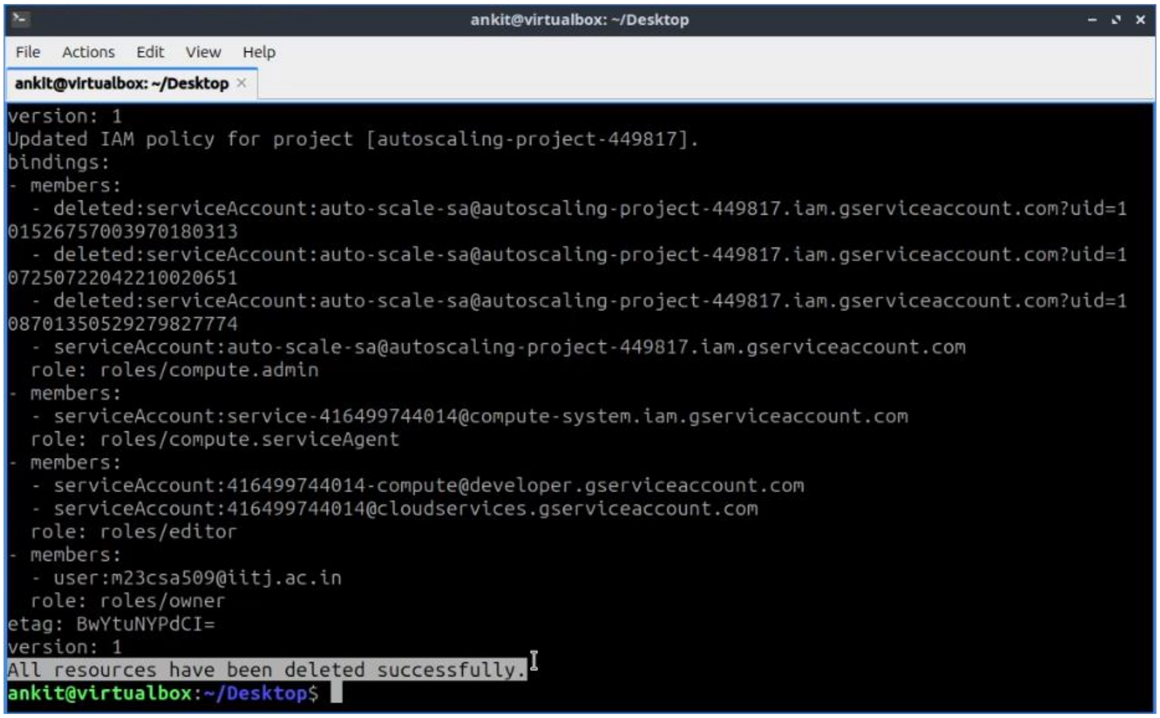

Fig. 17. All Resources deleted in GCP

3. Architecture Design of Infrastructure as a service (IaaS)

The architecture diagram in Fig. 18 illustrates:

- A **Managed Instance Group** dynamically scaling VM instances based on CPU utilization.
- An **Application Load Balancer** for distributing incoming traffic.
- **Firewall Rules** for controlling traffic.
- **IAM Roles** for secure access.

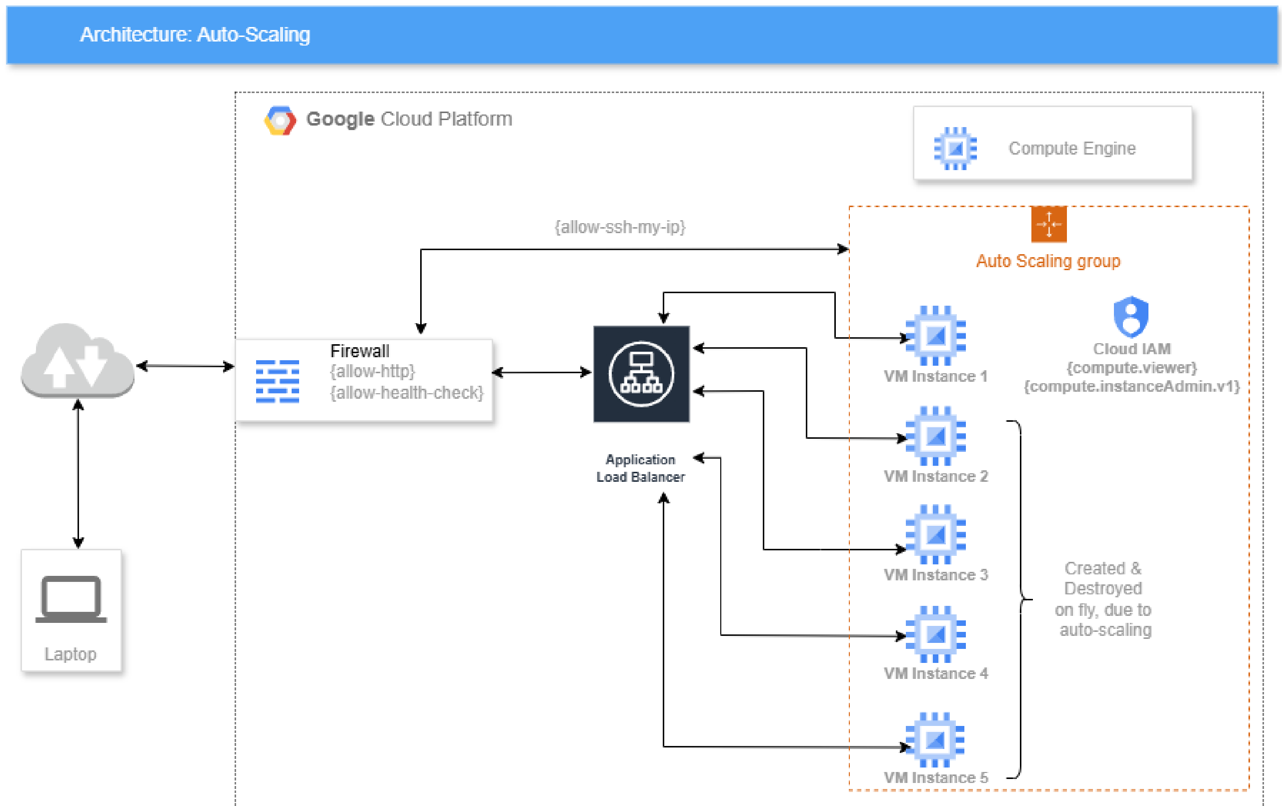

Fig. 18. Architecture of Infrastructure as a service (IaaS) on Google Cloud Platform

4. Link to Source Code Repository

The source codes used for this implementation are available at the following repository: [GitHub Repository Link]

Key Features of the Script are:

- Automates GCP SDK installation.
- Authenticates and configures GCP project settings.
- Deploys a managed instance group with auto-scaling policies.
- Configures security rules and load balancer settings.

5. Link to Recorded Video Demo

Here is a link to a recorded video [Link] demonstration of the setup process, which shows the auto-scaling and security configurations.

References:

- https://cloud.google.com/sdk?hl=en
- https://cloud.google.com/cli?hl=en
- https://cloud.google.com/compute/docs/autoscaler
- https://cloud.google.com/firewall/docs/firewalls
- https://cloud.google.com/iam/docs/overview